

手摸手之梳理数据库表关系

用户管理

会员数据

会员相关核心数据库表如下：

- t_user 会员数据表：存储会员账号、密码、证件号、邮箱、手机号等信息
- t_user_mail 会员邮箱数据表：存储会员邮箱和用户名的关系
- t_user_phone 会员手机号数据表：存储会员手机号和用户名的关系

很多小伙伴比较疑惑，和用户直接相关的三张表之间的关系是什么？手机号和邮箱不是可以在用户表中已经有了么，为什么还要再单独定义？

这是因为，我们对用户表进行了分库分表行为，分片键使用的 username 用户名进行拆分。

当我们执行新增语句时，没有分库分表前的数据长这个样子。

```
INSERT INTO `t_user` (`username`, `password`, `real_name`) VALUES ('admin', 'admin123456', '徐万里');
```

通过分库分表中间件 ShardingSphere 修饰后，真正执行的 SQL 语句长这个样子。

```
INSERT INTO `t_user_10` (`username`, `password`, `real_name`) VALUES ('admin', 'admin123456', '徐万里');
```

ShardingSphere 会通过分片键 username 用户名来确定数据在哪个库中的哪个表。所以，分库分表后，每次增删改查都需要带上分片键用户名。不然的话，查询会请求所有库的所有用户表，新增、修改和删除会直接报错。

我们以新增不带分片键根据手机号查询用户举例，真实执行的 SQL 如下：

```
SELECT id,username,password,real_name,region,id_type,id_card AS id_card,phone AS phone,telephone,mail AS mail,user_type,verify_status,post_code,address AS
```

```

address,deletion_time,create_time,update_time,del_flag  FROM t_user_0
WHERE del_flag=0

AND (phone = ?) UNION ALL SELECT id,username,password,real_name,region,id_type,id_card
AS id_card,phone AS phone,telephone,mail AS
mail,user_type,verify_status,post_code,address AS
address,deletion_time,create_time,update_time,del_flag  FROM t_user_1
WHERE del_flag=0

AND (phone = ?) UNION ALL SELECT id,username,password,real_name,region,id_type,id_card
AS id_card,phone AS phone,telephone,mail AS
mail,user_type,verify_status,post_code,address AS
address,deletion_time,create_time,update_time,del_flag  FROM t_user_2
WHERE del_flag=0

AND (phone = ?) UNION ALL SELECT id,username,password,real_name,region,id_type,id_card
AS id_card,phone AS phone,telephone,mail AS
mail,user_type,verify_status,post_code,address AS
address,deletion_time,create_time,update_time,del_flag  FROM t_user_3
WHERE del_flag=0

AND (phone = ?) UNION ALL SELECT id,username,password,real_name,region,id_type,id_card
AS id_card,phone AS phone,telephone,mail AS
mail,user_type,verify_status,post_code,address AS
address,deletion_time,create_time,update_time,del_flag  FROM t_user_4
WHERE del_flag=0
xxxxxxx

```

可以发现，ShardingSphere 通过 `UNION ALL` 的方式查询了所有分表。所以，这种不带分片键的查询在实际业务中，是不被允许的。

那问题来了，12306 支持会员用户登录时根据手机号、邮箱以及用户名三种组合方式登录，手机号和邮箱查询性能岂不是很差，因为要查询所有表。为了解决这个问题，我们通过路由表的方式解决这个问题。

相关技术实现方案查看下述文档的章节。

[手摸手之从创建会员接口开始](#)

用户相关扩展功能表如下：

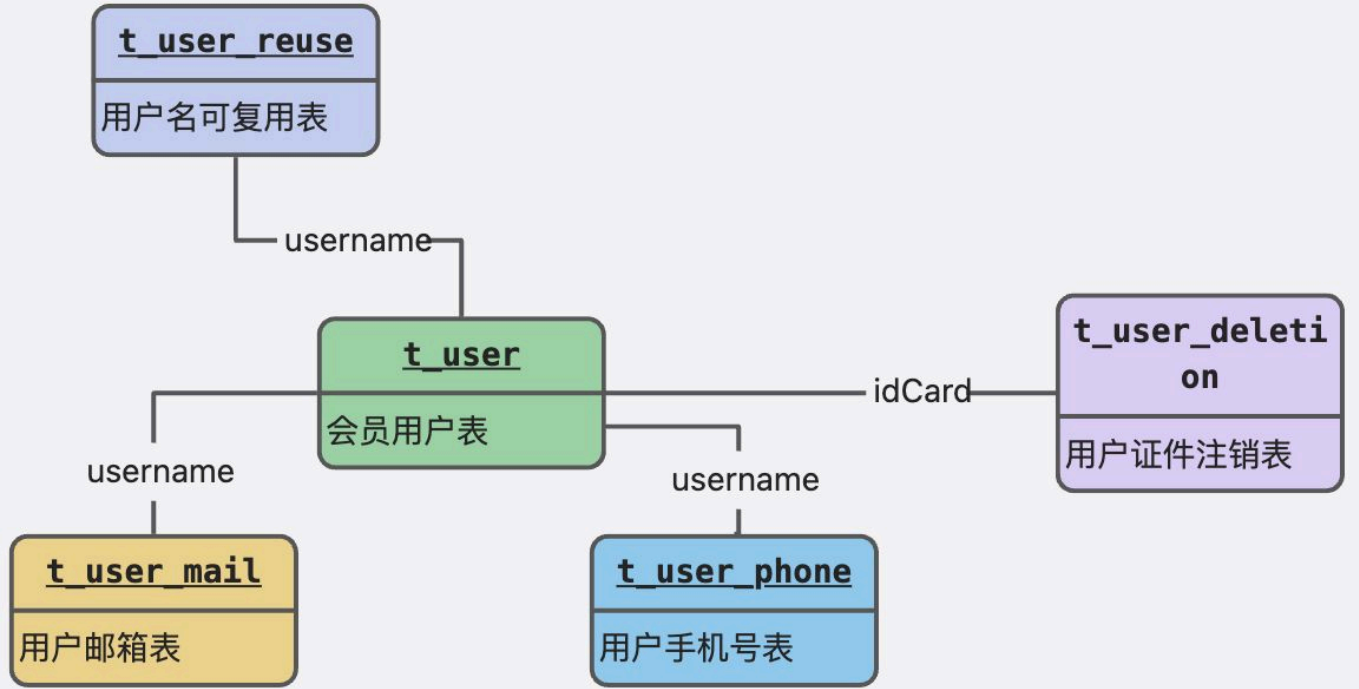
- t_user_reuse 用户名可复用表：存储已被注销的可用用户名
- t_user_deletion：用户证件号注销表：存储被注销过得证件号记录数据

t_user_reuse 用户名可复用表，用来解决布隆过滤器无法删除的问题，具体可查看以下文档的章节。

手摸手之从创建会员接口开始

t_user_deletion 用户证件号注销表，用来解决用户恶意注销 12306 账号的问题。假设，一个用户用一个证件号反复注册并注销，是不是应该把他拉入黑名单。因为用户名每次注册都可以使用新的，所以说我们只能针对证件号当做依据。目前系统的规则是，如果同一个证件号注销超过 5 次，就拉入黑名单不再支持注册。

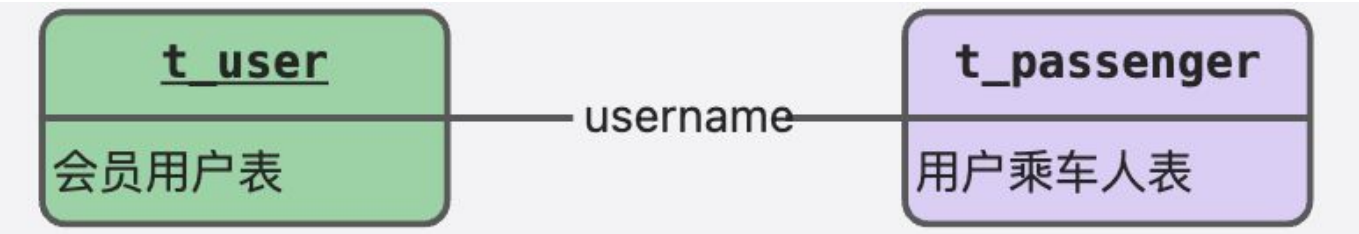
会员数据整体 UML 图如下所示。



乘车人数据

- t_passenger：乘车人数据表

一个用户可以多名乘车人，可以用来买票时选择多人购票。用户表和乘车人表之间通过用户名进行关联，一对多的关联关系。



配套视频

<<https://t.zsxq.com/108kS2rZe>>

列车管理

列车数据表介绍

t_train：列车表，存储每天生成的行驶列车数据。

北京北 --> 杭州 (8月9日 周三) 共计17个车次 您可使用 **中转换乘** 功能，查询途中换乘一次的部分列车余票情况。 ☐ 显示积分兑换车次 ☐ 显示全部可预订车次

车次	出发站 到达站	出发时间 到达时间	历时	商务座 特等座	一等座	二等座 二等包座	高级 软卧	软卧 一等卧	动卧	硬卧 二等卧	软座	硬座	无座	其他	备注
G33 	北京南 杭州东	08:56 13:28	04:32 当日到达	9	无	2	--	--	--	--	--	--	--	--	预订
G115 	北京南 杭州东	09:10 15:47	06:37 当日到达	1	无	无	--	--	--	--	--	--	--	--	预订
G169 	北京南 杭州东	09:38 15:38	06:00 当日到达	无	无	无	--	--	--	--	--	--	--	--	预订
G35 	北京南 杭州东	09:56 14:26	04:30 当日到达	1	无	无	--	--	--	--	--	--	--	--	预订

t_carriage：列车车厢表，存储每趟列车对应的车厢数据，包括车厢类型。

#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment
1	id	bigint(20) unsigned	NULL	NULL	NO	NULL	auto_increment	EMPTY	ID
2	train_id	bigint(20)	NULL	NULL	YES	NULL	EMPTY	EMPTY	列车ID
3	carriage_number	varchar(64)	utf8mb4	utf8mb4_unicode_ci	YES	NULL	EMPTY	EMPTY	车厢号
4	carriage_type	int(3)	NULL	NULL	YES	NULL	EMPTY	EMPTY	车厢类型
5	seat_count	int(3)	NULL	NULL	YES	NULL	EMPTY	EMPTY	座位数
6	create_time	datetime	NULL	NULL	YES	NULL	EMPTY	EMPTY	创建时间
7	update_time	datetime	NULL	NULL	YES	NULL	EMPTY	EMPTY	修改时间
8	del_flag	tinyint(1)	NULL	NULL	YES	NULL	EMPTY	EMPTY	删除标识

t_train_station：列车站点表，存储列车行驶站点顺序表。

北京北 --> 杭州 (8月9日 周三) 共计17个车次 您可使用 **中转换乘** 功能，查询途中换乘一次的部分列车余票情况

车次	出发站		出发时间 ▲	历时 ▲	商务座	一等座	二等座	高级
	站序	站名	到站时间	出发时间	停留时间 X		二等包座	软卧
G33 	01	北京南	09:38	09:38	----	无	2	--
G115	02	天津南	10:12	10:14	2分钟	无	无	--
	03	沧州西	10:37	10:45	8分钟			
G169	04	济南西	11:31	11:34	3分钟	无	无	--
	05	徐州东	12:40	12:43	3分钟			
G35 	06	宿州东	13:02	13:04	2分钟	无	无	--
G181	07	南京南	14:04	14:09	5分钟	候补	候补	--
	08	句容西	14:21	14:23	2分钟			
G37 	G169次		北京南 --> 南昌西	高速	有空调	候补	候补	--

t_train_station_relation: 列车站点关联表，存储列车行驶站点关联关系表。

这是一个会把所有车站都会关联起来的数据集合，存储的数据模型如下所示：

- 北京南->天津西，北京南->沧州西，北京南->xxx，北京南->句容西。
- 天津西->沧州西，天津西->济南西，天津西->xxx，天津西->句容西。
- 沧州西->济南西，沧州西->徐州东，沧州西->xxx，天津西->句容西。

北京北 --> 杭州 (8月9日 周三) 共计17个车次 您可使用中转换乘功能，查询途中换乘一次的部分列车余票情况

车次	出发站		出发时间		历时	商务座 特等座	一等座	二等座 二等包座	高级 软卧
	站序	站名	到站时间	出发时间					
G33 	01	北京南		08:00	8	----	无	2	--
G115	02	天津南		10:04	2分钟		无	无	--
	03	沧州西		10:05	8分钟				
	04	济南西		10:04	3分钟		无	无	--
G169	05	徐州东	12:00	12:03	3分钟		无	无	--
G35 	06	宿州东	13:02	13:04	2分钟		无	无	--
G181	07	南京南	14:04	14:09	5分钟		候补	候补	--
	08	句容西	14:21	14:23	2分钟				
G37 	G169次		北京南 --> 南昌西		高速	有空调	候补	候补	--

t_train_station_price: 列车站点价格表，存储列车站点关联关系不同座位价格表。

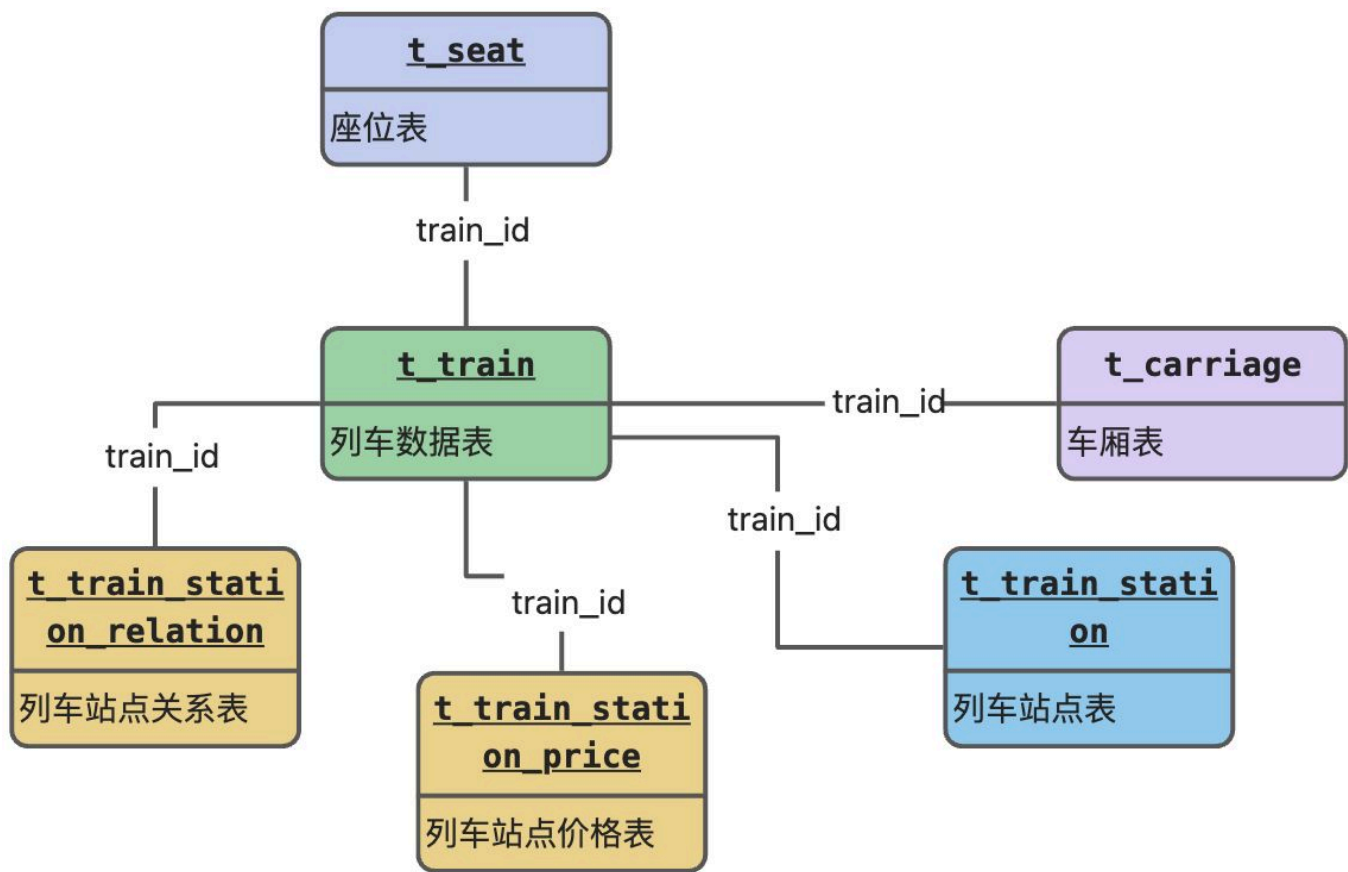
价格表在 t_train_station_relation 关系表的基础上，加入了两个字段，一个是价格，一个是座位类型。

因为列车行驶路线中，不同的站点间隔费用不一样，而且不同的座位费用也不一样。通过价格表整体区分出来。

北京北 --> 杭州 (8月17日 周四) 共计23个车次 您可使用中转换乘功能，查询途中换乘一次的部分

车次	出发站 到达站	出发时间 到达时间	历时	商务座 特等座	一等座	二等座 二等包座
G171 	北京南 杭州东	06:30 12:20	05:50 当日到达	3	有	有
				¥2016.0	¥991.0	¥594.0

表关联关系图



配套视频

<<https://t.zsxq.com/11NLBgw01>>

订单管理

订单数据表介绍

t_order: 订单主表，用户购买的单次车票，就对应一个订单。但是有可能一个订单中会有多个乘车人，所以还会有订单明细表。

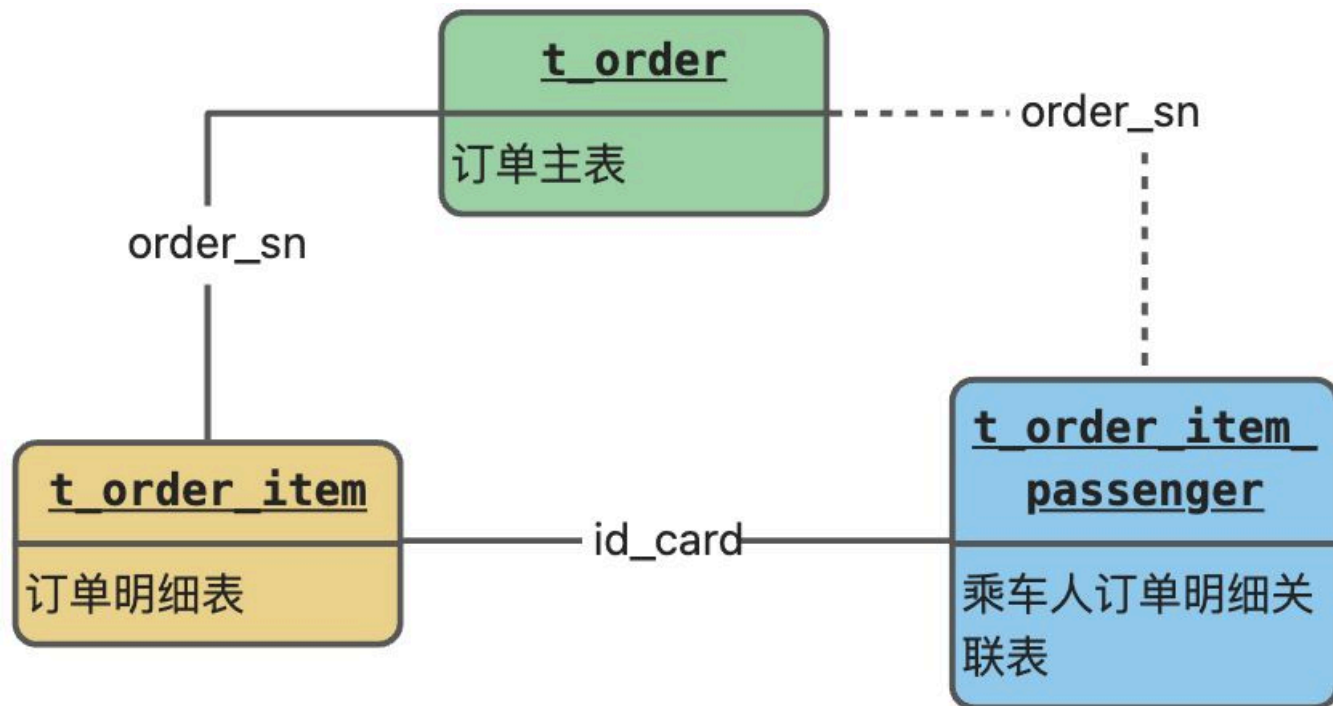
t_order_item: 订单明细表，一个订单可能有多个乘车人，多个乘车人就对应多个订单明细。

订票日期: 2023-08-08 北京南 --> 杭州东 G35 2023-06-01 09:56 开	☐ 万重山 打印信息单 中国居民身份证	商务座 01车01F号	成人票 ¥2313
	☐ 金来 打印信息单 中国居民身份证	商务座 01车02F号	成人票 ¥2313
	☐ laosan 打印信息单 中国居民身份证	商务座 16车01F号	成人票 ¥2313

t_order\item\passenger: 订单明细乘车人表, 因为订单表和订单明细表分库分表规则所致, 乘车人无法查看本人车票订单, 所以创建了这个关联表。通过证件号关联订单。

Q: 为什么通过乘车人证件号关联?

A: 因为用户在添加乘车人进行购票时, 有可能乘车人是没有注册账号的, 所以只能通过证件号进行购票。那如果后来用户注册了账号, 也就只能通过证件号去关联自己本人车票订单。



配套视频

<<https://t.zsxq.com/112oq1bXc>>

支付管理

t_pay: 订单支付表, 存储用户支付车票相关数据。

t_pay

支付表

更新: 2023-08-23 14:13:33

原文: <<https://www.yuque.com/magestack/12306/ihi1rsmxsl5fq0u4>>